

DESIGN PLAN · IMPLEMENTATION BLUEPRINT

AlumniConnect

Alumni Engagement & Career Referral Platform

Centralized platform for a single college

Platform: Web (Next.js) + Mobile (React Native) + Backend (Node.js/Express) + PostgreSQL

Goal: Real production deployment for a college

Timeline: 8-week phased delivery · **Scope:** V1 feature set

1 Project Overview

4

USER ROLES

6

CORE MODULES

8

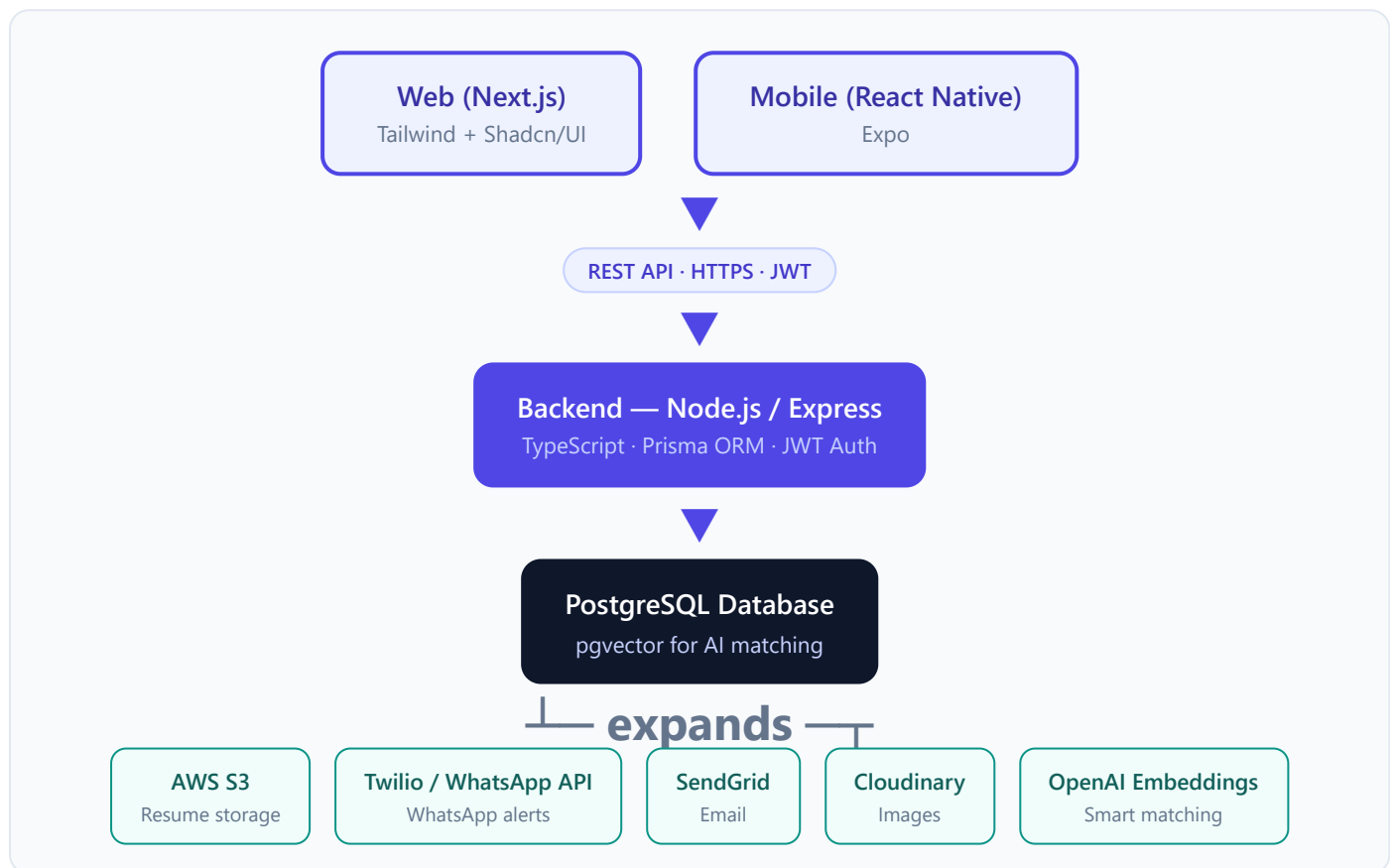
WEEK PLAN

6

TECH LAYERS

AlumniConnect is a centralized engagement & career-referral platform for a single college. It connects **students** with **verified alumni** for mentorship, job referrals, and success-story inspiration — delivered through web, mobile, and notification channels (email + WhatsApp).

2 High-Level Architecture



Hosting: Vercel (web) · Railway/Render (backend) · Supabase (DB). All integrations plug in behind the API — swappable at deployment time.

3 User Roles

Role	Who	Capabilities	Badge
Super Admin	College IT / Admin	Manage everything, CSV import, verify alumni, approve stories	Admin
Alumni	Graduated students	Edit profile, post jobs, refer candidates, mentor students	Alumni
Student	Current students	Browse directory, request referrals, apply to jobs	Student
Faculty	Teachers	View directory, post announcements	Faculty

Role ↔ Feature Permission Matrix

Feature	Super Admin	Alumni	Student	Faculty
Alumni Directory (search/filter)	✓	✓	✓	✓
Edit personal profile	✓	✓	✓	—
Post jobs	✓	✓	—	—
Request referral	—	—	✓	—
Approve/reject referrals	✓	✓	—	—
Post announcements	✓	—	—	✓
CSV bulk import	✓	—	—	—
Approve success stories	✓	—	—	—

4 Database Schema — Core Tables

User id, name, email, password_hash, role, phone, avatar college_id, batch_year, department, current_company, job_title location, linkedin_url, bio, is_verified · created_at, updated_at
JobPosting id, posted_by (user_id), company, role, location description, requirements, apply_link, deadline referral_slots, status (open/closed) · created_at
ReferralRequest id, job_id, requested_by (student_id), referred_by (alumni_id) resume_url, cover_letter, status (pending/accepted/rejected/hired) notes, referral_date · created_at, updated_at

Event → EventRSVP (user_id, event_id)

id, title, description, date, location, mode (online/offline)
created_by, cover_image, rsvp_count

SuccessStory

id, alumni_id, title, story, company, role, batch_year
image_url, is_featured, approved_by_admin · created_at

Notification

id, user_id, type, title, message, link, is_read · created_at

5 Core Features (V1)

🕒 A. Alumni Directory & Profile

- Search / filter by batch year, department, company, location
- Profile pages with career timeline
- Privacy controls — some info visible only to verified alumni
- AI Smart Matching: "Top 5 Alumni for You" by student interests

✉ B. Job Board & Referrals

- Alumni post job openings at their company
- Students browse + send referral requests (resume upload)
- Alumni dashboard: accept/reject referral requests
- Status tracking: Pending → Accepted → Referred → Hired
- In-app + email + WhatsApp notifications

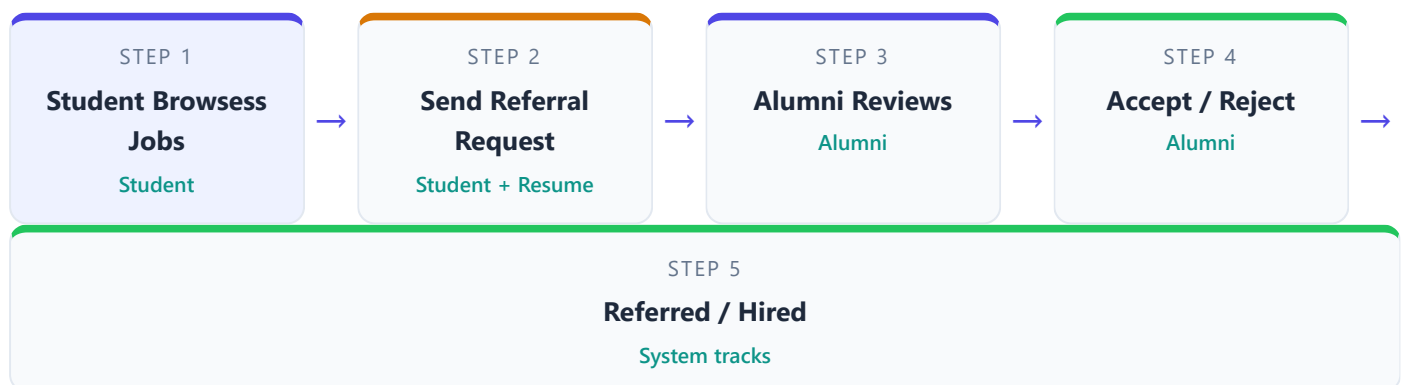
★ C. Success Stories / Spotlight Wall

- Alumni submit success stories
- Admin approval workflow
- Public gallery on homepage

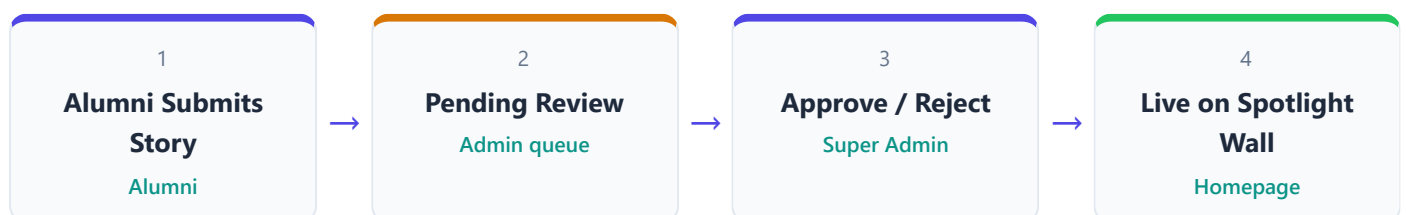
↑ D. Bulk CSV Import (Admin)

- Admin uploads CSV of alumni data
- System parses, validates, creates accounts
- Sends welcome email with activation link

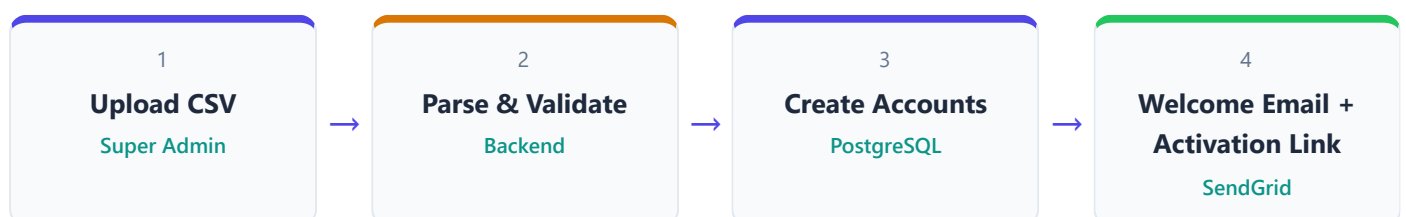
Referral Workflow



Success Story Approval Workflow



CSV Import Workflow (Admin)



6 Tech Stack (Final)

Layer	Technology
Frontend Web	Next.js 14 · Tailwind CSS · Shadcn/UI
Mobile	React Native + Expo
Backend API	Node.js · Express · TypeScript
Database	PostgreSQL + Prisma ORM
Auth	JWT + bcrypt + Google OAuth (optional)
File Storage	AWS S3 / Cloudinary (resumes, images)
Email	SendGrid
WhatsApp	Twilio / WhatsApp Business API
AI Matching	OpenAI Embeddings + pgvector
Hosting	Vercel (web) · Railway/Render (backend) · Supabase (DB)

Development Phases — 8-Week Roadmap

Foundation	Directory	Jobs & Referrals	Engagement	AI & Polish	Deploy
------------	-----------	------------------	------------	-------------	--------

W1-2 W3 W4-5 W6 W7 W8

Phase 1 · Foundation WEEK 1-2	- Setup repo, Next.js + Express boilerplate - PostgreSQL schema with Prisma - JWT auth (register, login, logout) - User roles & basic profile
Phase 2 · Alumni Directory WEEK 3	- Profile CRUD · Search + filters · Profile pages - CSV bulk import for admin
Phase 3 · Jobs & Referrals WEEK 4-5	- Job posting form · Job listing + detail pages - Referral request flow · Status tracking dashboard
Phase 4 · Engagement WEEK 6	- Success stories wall - Notification system (in-app + email) · WhatsApp alerts

Phase 5 · AI & Polish
WEEK 7

- AI Smart Matching ("Top 5 Alumni for You")
- Admin dashboard with analytics
- Mobile app (React Native) — read-only views + referrals

Phase 6 · Deployment
WEEK 8

- Deploy web (Vercel) · Backend (Railway/Render) · Mobile (Expo)
- Test with real college data · Documentation

7 API Endpoints (Sample)

```
# Auth & Users
POST /api/auth/register
POST /api/auth/login
GET /api/users/me
GET /api/alumni?batch=2018&dept=CSE&company=Google
GET /api/alumni/:id

# Jobs & Referrals
POST /api/jobs # alumni only
GET /api/jobs
POST /api/referrals # student → alumni
PATCH /api/referrals/:id/status # alumni only
GET /api/referrals/me

# AI Matching
GET /api/match/top-alumni # AI matching

# Stories & Admin
POST /api/stories
GET /api/stories?featured=true
POST /api/admin/import-csv
```

8 Suggested Folder Structure

```
alumni-platform/
├── apps/
│   ├── web/ # Next.js
│   ├── mobile/ # React Native
│   └── api/ # Express backend
├── packages/
│   ├── shared-types/ # TypeScript types
│   └── ui/ # Shared components
├── prisma/
│   └── schema.prisma
└── docs/
```

Next Steps — Priority Order

1. Set up the project repo (monorepo with web + API)
2. Design the database schema in detail (Prisma)
3. Build the auth system first
4. Create wireframes / mockups for the UI
5. Plan the AI matching algorithm